how to split the middleware pipeline into branches, and you'll find out when this is useful.

The middleware pipeline is one of the building blocks of ASP.NET Core apps, so we covered it in depth in chapter 4. Every request passes through the middleware pipeline, and each middleware component in turn gets an opportunity to modify the request or to handle it and return a response. ASP.NET Core includes middleware for handling common scenarios out of the box. You'll find middleware for serving static files, handling errors, authentication, and many more tasks.

You'll spend most of your time during development working with Razor Pages, minimal API endpoints, or web API controllers. These are exposed as the endpoints for most of your app's business logic, and they call methods on your app's various business services and models. However, you've also seen middleware like the Swagger middleware and the `WelcomePageMiddleware` that returns a response without using the endpoint routing system. The various improvements to the routing system in .NET 7 mean I rarely find the need to create "terminal" middleware like this, as endpoint routing is easy to work with and extensible. Nevertheless, it may occasionally be preferable to create small, custom, terminal middleware components like these.

At other times, you might have requirements that lie outside the remit of Razor Pages or minimal API endpoints. For example, you might want to ensure that all responses generated by your app include a specific header. This sort of cross-cutting concern is a perfect fit for custom middleware. You could add the custom middleware early in your middleware pipeline to ensure that every response from your app includes the required header, whether it comes from the static-file middleware, the error handling middleware, or a Razor Page.